

```

import requests
import json
from datetime import datetime

KALSHI_BASE = "https://external-api.kalshi.com/trade-api/v2"
NTFY_TOPIC = "free-money-alert"
FEE_RATE = 0.03
PAYOUT_AFTER_FEE = 1 - FEE_RATE # $0.97 guaranteed payout per contract pair
MIN_EDGE_PCT = 0.01 # 1% minimum edge
MAX_SPEND = 500 # max dollars per trade
MIN_SPEND = 50 # min spend worth alerting on

def fetch_markets():
    all_markets = []
    cursor = None
    for _ in range(10):
        params = {"status": "open", "limit": "100"}
        if cursor:
            params["cursor"] = cursor
        try:
            res = requests.get(f"{KALSHI_BASE}/markets", params=params, timeout=1)
            res.raise_for_status()
            data = res.json()
            batch = data.get("markets", [])
            all_markets.extend(batch)
            cursor = data.get("cursor")
            if not cursor or len(batch) < 100:
                break
        except Exception as e:
            print(f"Error fetching markets: {e}")
            break
    return all_markets

def analyze(markets):
    opportunities = []
    for m in markets:
        try:
            yes_ask = float(m.get("yes_ask_dollars", 0))
            no_ask = float(m.get("no_ask_dollars", 0))
            liquidity = float(m.get("liquidity_dollars", 0))
        except (TypeError, ValueError):
            continue

        if yes_ask <= 0 or no_ask <= 0:

```

```

        continue

combined_cost = yes_ask + no_ask
edge_per_contract = PAYOUT_AFTER_FEE - combined_cost

if edge_per_contract <= 0:
    continue

edge_pct = edge_per_contract / combined_cost
if edge_pct < MIN_EDGE_PCT:
    continue

# How many contracts can you buy with MAX_SPEND?
contracts_max = int(MAX_SPEND / combined_cost)
# How many contracts needed to make $50 minimum?
contracts_min_profit = int(50 / edge_per_contract) + 1

if contracts_min_profit > contracts_max:
    continue # can't hit $50 profit within MAX_SPEND

# Cap by liquidity too
contracts_by_liq = int(liquidity / combined_cost) if liquidity > 0 else c
contracts = min(contracts_max, contracts_by_liq)

if contracts < contracts_min_profit:
    continue # not enough liquidity to make it worth it

total_spend = round(contracts * combined_cost, 2)
total_payout = round(contracts * PAYOUT_AFTER_FEE, 2)
total_profit = round(total_payout - total_spend, 2)

if total_spend < MIN_SPEND:
    continue

close_time = m.get("close_time", "")
try:
    close_dt = datetime.fromisoformat(close_time.replace("Z", "+00:00"))
    from datetime import timezone
    diff = close_dt - datetime.now(timezone.utc)
    days = diff.days
    hours = diff.seconds // 3600
    closes_in = f"{days}d {hours}h" if days > 0 else f"{hours}h {(diff.se
except:
    closes_in = "unknown"

opportunities.append({
    "ticker": m.get("ticker", ""),

```

```

        "event": m.get("event_ticker", ""),
        "title": m.get("yes_sub_title") or m.get("ticker", ""),
        "yes_ask": yes_ask,
        "no_ask": no_ask,
        "combined_cost": combined_cost,
        "edge_per_contract": edge_per_contract,
        "edge_pct": edge_pct,
        "contracts": contracts,
        "total_spend": total_spend,
        "total_profit": total_profit,
        "liquidity": liquidity,
        "closes_in": closes_in,
    })

# Sort by total profit descending
opportunities.sort(key=lambda x: x["total_profit"], reverse=True)
return opportunities

def send_notification(opp):
    title = f"💰 KALSHI ARB - ${opp['total_profit']:.2f} PROFIT"

    body = (
        f"Market: {opp['title']}\n"
        f"Ticker: {opp['ticker']}\n\n"
        f"—— THE TRADE ——\n"
        f"Buy {opp['contracts']} YES contracts @ ${opp['yes_ask']:.2f} each\n"
        f"Buy {opp['contracts']} NO contracts @ ${opp['no_ask']:.2f} each\n\n"
        f"—— THE MATH ——\n"
        f"Total spend: ${opp['total_spend']:.2f}\n"
        f"Guaranteed payout: ${opp['contracts'] * PAYOUT_AFTER_FEE:.2f}\n"
        f"Profit: ${opp['total_profit']:.2f} ({opp['edge_pct']*100:.2f}% edge)\n"
        f"—— HOW TO EXECUTE ——\n"
        f"1. Open Kalshi app or kalshi.com\n"
        f"2. Search ticker: {opp['ticker']}\n"
        f"3. Buy {opp['contracts']} YES @ ${opp['yes_ask']:.2f} (limit order)\n"
        f"4. Buy {opp['contracts']} NO @ ${opp['no_ask']:.2f} (limit order)\n"
        f"5. Wait for market to resolve - collect ${opp['contracts'] * PAYOUT_AFT
        f"Liquidity: ${opp['liquidity']:.0f} | Closes in: {opp['closes_in']}\n"
        f"⚠ Place BOTH orders fast - opportunity closes in seconds"
    )

    try:
        requests.post(
            f"https://ntfy.sh/{NTFY_TOPIC}",
            data=body.encode("utf-8"),
            headers={
                "Title": title,

```

```

        "Priority": "high",
        "Tags": "money_with_wings,chart_with_upwards_trend",
        "Click": f"https://kalshi.com/markets/{opp['event']}",
    },
    timeout=10,
)
    print(f"Notification sent: {opp['ticker']} - ${opp['total_profit']:.2f} p
except Exception as e:
    print(f"Failed to send notification: {e}")

def main():
    print(f"[{datetime.now().strftime('%H:%M:%S')}] Scanning Kalshi markets...")
    markets = fetch_markets()
    print(f"Fetched {len(markets)} markets")
    opps = analyze(markets)
    print(f"Found {len(opps)} opportunities")

    if not opps:
        print("No arb found this scan.")
        return

    for opp in opps[:3]: # notify top 3 opportunities max per scan
        send_notification(opp)

if __name__ == "__main__":
    main()

```